Application of Static Analysis (2)

CSE552 Program Analysis — Lecture 13

Jaemin Hong

Output Parameters in C

- An output parameter is a pointer-type parameter used for producing output, rather than for taking input
- Widely used in C to implement
  - Functions that return multiple values
  - Partial functions (functions that may fail)

Multiple Return Values

```
int div(int n, int m, int *q) {
  *q = n / m;
  return n % m;
}
```

- The quotient is written to the output parameter q
- The remainder is produced as the return value

Partial Functions

```
int div(int n, int m, int *q) {
  if (m == 0) {
    return -1;
  }
  *q = n / m;
  return 0;
}
```

- The quotient is written to the output parameter q
- The return value signifies success or failure

Disadvantages of Output Parameters

- Output parameters do not directly violate memory safety, but undermine readability
  - Parameters are primarily intended for input, not output
  - Pointer-type parameters can be used for input, returning multiple values, or partial functions, which can be confusing
- Partial functions implemented with output parameters are especially error-prone
  - Programmers may forget to check the return value

Example

```
int div(int n, int m, int *q) {
  if (m == 0) { return -1; }
  *q = n / m;
  return 0;
}
```

Good:

```
if (div(n, m, &x) == 0) {
  // use x
}
```

Bad:

```
div(n, m, &x);
// use x
```

ADTs in Rust

- Among diverse algebraic data types (ADTs), tuples, `Option`, and `Result` are especially useful
  - Multiple return values and partial functions can be implemented with these types

Multiple Return Values

```
fn div(n: i32, m: i32) -> (i32, i32) {
  (n / m, n % m)
}
```

- The quotient and the remainder are returned as a tuple

Partial Functions with Option

```
fn div(n: i32, m: i32) -> Option<i32> {
  if m == 0 {
    None
  } else {
    Some(n / m)
  }
}
```

- The quotient is returned as an `Option`
- `None` signifies failure, while `Some` contains the quotient on success

Partial Functions with Result

```
fn div(n: i32, m: i32) -> Result<i32, String> {
  if m == 0 {
    Err(String::from("division by zero"))
  } else {
    Ok(n / m)
  }
}
```

- The quotient is returned as a `Result`
- `Err` contains an error message on failure, while `Ok` contains the quotient on success

Advantages of ADTs

- ADTs improve readability by conveying the intent with return types
  - Tuples express the intent of returning multiple values
  - `Option` and `Result` express the possibility of failure
- Partial functions implemented with `Option` or `Result` are less error-prone
  - The compiler requires programmers to explicitly handle the possibility of failure

Example

```
fn div(n: i32, m: i32) -> Option<i32> {
  if m == 0 { None } else { Some(n / m) }
}

match div(n, m) {
  None => {
    // handle failure
  }
  Some(x) => {
    // use x
  }
}
```

- To obtain the quotient, programmers must use pattern matching, which forces them to handle the failure case

Translating Output Parameters to ADTs

- As ADTs can be used instead, output parameters are considered unidiomatic in Rust and discouraged
- Ideal C-to-Rust translators should translate output parameters to direct return of ADTs

Challenges

- Identifying output parameters
  - Not every pointer-type parameter is an output parameter
- Classifying output parameters
  - Whether failure is possible or not
- Inferring the meaning of the return value
  - Whether it signifies success or failure
- Can be solved with static analysis

Static Analysis for Output Parameters

- Bottom-up modular analysis
  - Each function is analyzed once to create a function summary
  - A callee is analyzed before its caller is analyzed
  - Analyzing a caller requires the callees' function summaries

Abstract Read/Write Sets

- Abstract read set: $R = (\mathcal{P}(Param), \subseteq)$
  - Pointer-type parameters that may have been read before being written to while reaching the current program point
- Abstract write set: $W = (\mathcal{P}(Param), \supseteq)$
  - Pointer-type parameters that must have been written to before being read while reaching the current program point
- Intuition: parameters contained in the abstract write set at the function return are output parameters

Analysis State and Join

- $State = R \times W$
- $JOIN(v) = \bigsqcup_{u \in pred(v)} [\![u]\!]$
  - Forward analysis

Constraint Rules

Let $JOIN(v) = (r, w)$.

- entry: $[\![v]\!] = (\emptyset, \emptyset)$
- `y = *x`:

$$[\![v]\!] = \begin{cases} (r \cup \{x\}, w) & \text{if } x \notin w \\ (r, w) & \text{otherwise} \end{cases}$$

- `*x = e`:

$$[\![v]\!] = \begin{cases} (r, w \cup \{x\}) & \text{if } x \notin r \\ (r, w) & \text{otherwise} \end{cases}$$

Constraint Rules — Example 1

```
void foo(int *p) {
  // r = {}, w = {}
  *p = 42;
  // r = {}, w = {p}
}
```

- p is an output parameter

Constraint Rules — Example 2

```
void foo(int *p) {
  // r = {}, w = {}
  int x = *p;
  // r = {p}, w = {}
  *p = x + 1;
  // r = {p}, w = {}
}
```

- p is not an output parameter
  - Without taking p, the function cannot decide the value of x

Constraint Rules — Example 3

```
int foo(int *p) {
  // r = {}, w = {}
  *p = 42;
  // r = {}, w = {p}
  int x = *p;
  // r = {}, w = {p}
  return x;
  // r = {}, w = {p}
}
```

- p is an output parameter
  - The function can decide the value of x even without taking p

Constraint Rules — Example 4

```
void foo(int *p) {
  // r = {}, w = {}
  if (...) {
    // r = {}, w = {}
    *p = 42;
    // r = {}, w = {p}
  } else {
    // r = {}, w = {}
    int x = *p;
    // r = {p}, w = {}
  }
  // r = {p}, w = {}
}
```

- p is not an output parameter
  - Without taking p, the function cannot decide the value of x

Constraint Rules — Example 5

```
int foo(int *p) {
  // r = {}, w = {}
  int x = -1;
  // r = {}, w = {}
  if (...) {
    // r = {}, w = {}
    *p = 42;
    // r = {}, w = {p}
    x = 0;
    // r = {}, w = {p}
  }
  // r = {}, w = {}
  return x;
}
```

- p is an output parameter, but the abstract write set at the return is empty

Write Set Sensitivity

- $State = W \to R$
- Distinguish different write sets

Write Set Sensitivity — Example

```
int foo(int *p) {
  // {}: {}
  int x = -1;
  // {}: {}
  if (...) {
    // {}: {}
    *p = 42;
    // {}: {}
    x = 0;
    // {p}: {}
  }
  // {}: {}
  // {p}: {}
  return x;
}
```

- p is an output parameter, and failure is possible

Integer Set Domain

- Set domain: $V = (\{A \mid A \subseteq \mathbb{Z} \wedge |A| \leq k\} \cup \{\mathbb{Z}\}, \subseteq)$
  - An abstract value is a set of at most k integers, or the set of all integers
- $State = W \to (R \times (Var \to V))$
  - Can decide return values associated with different write sets

Integer Set Domain — Example 1

```
int foo(int *p) {
  // {}: {}, [x: {}]
  int x = -1;
  // {}: {}, [x: {-1}]
  if (...) {
    // {}: {}, [x: {-1}]
    *p = 42;
    // {}: {}, [x: {-1}]
    x = 0;
    // {p}: {}, [x: {0}]
  }
  // {}: {}, [x: {-1}]
  // {p}: {}, [x: {0}]
  return x;
}
```

- On failure, -1 is returned
- On success, 0 is returned

Integer Set Domain — Example 2

```
int foo(int *p) {
  int x = 0;
  if (...) {
    x = -1;
  } else if (...) {
    x = 2;
  } else if (...) {
    x = 5;
  } else {
    *p = 42;
  }
  return x;
}
```

- -1, 2, and 5 signify failure, while 0 signifies success
- The set domain gives $\{-1, 2, 5\}$ and $\{0\}$ for the two write sets, respectively
- The interval domain would give $[-1, 5]$ and $[0, 0]$, which have intersection, making it impossible to distinguish success and failure cases

Interpreting Analysis Results

- At return, if every write set contains `p`, then `p` is a must-output parameter
  - No failure case is possible
  - Replaced with direct return of a tuple
- At return, if some write set contains `p` and no read set contains `p`, then `p` is a may-output parameter
  - Failure cases are possible
  - If the success return value is unique and different from failure return values, replaced with direct return of a `Result`
  - Otherwise, replaced with direct return of an `Option`

Translation — Example 1

```
int foo(int *p) {
  *p = 42;
  return 0;
}
```

$\Rightarrow$

```
fn foo() -> (i32, i32) {
  (0, 42)
}
```

p is a must-output parameter

Translation — Example 2

```
int foo(int *p) {
  if (...) {
    *p = 42;
    return 0;
  } else {
    return -1;
  }
}
```

$\Rightarrow$

```
fn foo() -> Result<i32, i32
    > {
  if ... {
    Ok(42)
  } else {
    Err(-1)
  }
}
```

p is a may-output parameter, 0 signifies success, and -1 signifies failure

Translation — Example 3

```
int foo(int *p) {
  if (...) {
    *p = 42;
    return 0;
  } else {
    return 0;
  }
}
```

⇒

```
fn foo() -> (i32, Option<
    i32>) {
  if ... {
    (0, Some(42))
  } else {
    (0, None)
  }
}
```

p is a may-output parameter, and 0 can signify both success and failure

I/O in C

- I/O functions in C are not memory-safe
  - e.g., they do not check array bounds

```
FILE *fp = ...;
char buf[10];
fread(buf, 20, 1, fp); // buffer overflow
```

I/O in Rust

- I/O functions in Rust are memory-safe
  - e.g., they check array bounds

```
let mut f = ...;
let mut buf = [0u8; 10];
f.read(&mut buf[..]); // read up to 10 bytes
```

I/O Streams

- Streams can have different origins
  - Standard input/output/error, files
- Streams can have different capabilities
  - Readable, writable, seekable

Streams in C

- Every stream has the same type `FILE *` regardless of its origin and capabilities

```
FILE *fp = stdout;
fprintf(fp, "Hello, world!\n");

fp = fopen("file.txt", "r");
char buf[4];
fread(buf, 4, 1, fp);
```

Streams in C — Problems

- Streams may be used for operations that they do not support
  - Such operations do not violate memory safety but fail at runtime

```
FILE *fp = stdin;
fprintf(fp, "Hello, world!\n");
```

- Streams may be used after they have been closed
  - Undefined behavior

```
fclose(fp);
fprintf(fp, "Hello, world!\n");
```

Streams in Rust — Origins

- Different types represent different origins

```
let f0: Stdin = stdin();
let f1: Stdout = stdout();
let f2: Stderr = stderr();
let f3: File = File::open("file.txt").unwrap();
```

Streams in Rust — Capabilities

- Different traits represent different capabilities

```
fn f<R: Read>(r: R) {
  let mut buf = [0u8; 4];
  r.read(&mut buf);
}
fn g<W: Write>(w: W) {
  w.write(b"Hello, world!\n");
}
fn h<S: Seek>(s: S) {
  s.seek(SeekFrom::Start(0));
}
```

Streams in Rust — Type/Trait Mapping

- Each type implements the traits corresponding to its capabilities
  - `Stdin` implements `Read`
  - `Stdout` and `Stderr` implement `Write`
  - `File` implements `Read`, `Write`, and `Seek`

Streams in Rust — Advantages

- Some unsupported operations are prevented at compile time

```
let f: Stdin = stdin();
f.write(b"Hello, world!\n"); // compile-time error
```

- The ownership system prevents the use of closed streams

```
drop(f);
f.write(b"Hello, world!\n"); // compile-time error
```

Translating C I/O to Rust I/O

- Can ensure memory safety
- Less error-prone by preventing unsupported operations
- Prevent the use of closed streams
- Challenge: deciding the proper type for each stream
  - Can be solved with static analysis

Origin and Capability Analysis

- For each program variable x of type `FILE *`, we introduce constraint variables $[\![x]\!]_o$ and $[\![x]\!]_c$
  - $[\![x]\!]_o$ denotes the set of possible origins of x
  - $[\![x]\!]_c$ denotes the set of capabilities required for x
- $\mathit{Origins} = \{\mathit{stdin}, \mathit{stdout}, \mathit{stderr}, \mathit{file}\}$
- $\mathit{Capabilities} = \{\mathit{read}, \mathit{write}, \mathit{seek}\}$

Constraint Rules — Construction and Operations

- Stream constructions give origin constraints

$$\texttt{x = stdin} : \mathit{stdin} \in [\![x]\!]_o$$
$$\texttt{x = stdout} : \mathit{stdout} \in [\![x]\!]_o$$
$$\texttt{x = stderr} : \mathit{stderr} \in [\![x]\!]_o$$
$$\texttt{x = fopen(...)} : \mathit{file} \in [\![x]\!]_o$$

- Stream operations give capability constraints

$$\texttt{fgetc(x)} : \mathit{read} \in [\![x]\!]_c$$
$$\texttt{fputc(\_, x)} : \mathit{write} \in [\![x]\!]_c$$
$$\texttt{fseek(x, \_, \_)} : \mathit{seek} \in [\![x]\!]_c$$

Constraint Rules — Assignment

- Constraints are propagated through assignments

$$\texttt{x = y} : [\![x]\!]_o \supseteq [\![y]\!]_o \ \wedge \ [\![x]\!]_c \subseteq [\![y]\!]_c$$

- Any origins possible for y are also possible for x
- Any capabilities required for x are also required for y
  - Analogy: subtyping (a type with more methods is a subtype)

Solving Constraints

- Each constraint has one of the following forms:
  - $t \in x$
  - $x \subseteq y$
- We can use the cubic algorithm

Example — Constraints

```
x = fopen(...);
y = stdin;
if (...) {
  z = x;
} else {
  z = y;
}
fgetc(z);
```

- $file \in [\![x]\!]_o$
- $stdin \in [\![y]\!]_o$
- $[\![x]\!]_o \subseteq [\![z]\!]_o$, $[\![z]\!]_c \subseteq [\![x]\!]_c$
- $[\![y]\!]_o \subseteq [\![z]\!]_o$, $[\![z]\!]_c \subseteq [\![y]\!]_c$
- $read \in [\![z]\!]_c$

Example — Solution

- $[\![x]\!]_o = \{file\}$, $[\![x]\!]_c = \{read\}$
- $[\![y]\!]_o = \{stdin\}$, $[\![y]\!]_c = \{read\}$
- $[\![z]\!]_o = \{file, stdin\}$, $[\![z]\!]_c = \{read\}$

Translation Rules

- If a stream has a single origin, use the type corresponding to that origin
- If a stream has multiple origins, use the trait corresponding to the required capabilities

Translation Example

```
x = fopen(...);
y = stdin;
if (...) {
  z = x;
} else {
  z = y;
}
fgetc(z);
```

$\Rightarrow$

```
let x: File;
let y: Stdin;
let z: Box<dyn Read>;

x = File::open(...).unwrap
    ();
y = stdin();
if ... {
  z = Box::new(x);
} else {
  z = Box::new(y);
}
z.read(...);
```

- $[\![x]\!]_o = \{file\}$, $[\![x]\!]_c = \{read\}$
- $[\![y]\!]_o = \{stdin\}$, $[\![y]\!]_c = \{read\}$
- $[\![z]\!]_o = \{file, stdin\}$,
  $[\![z]\!]_c = \{read\}$

Summary

- Output parameters in C can be replaced with tuples, `Option`, and `Result` in Rust
- Analysis tracks abstract read/write sets and integer values with write-set sensitivity, which distinguishes different write sets
- C streams have the same type `FILE *`, while Rust has different types and traits for different origins and capabilities
- Origin and capability analysis solves membership/subset constraints with the cubic algorithm